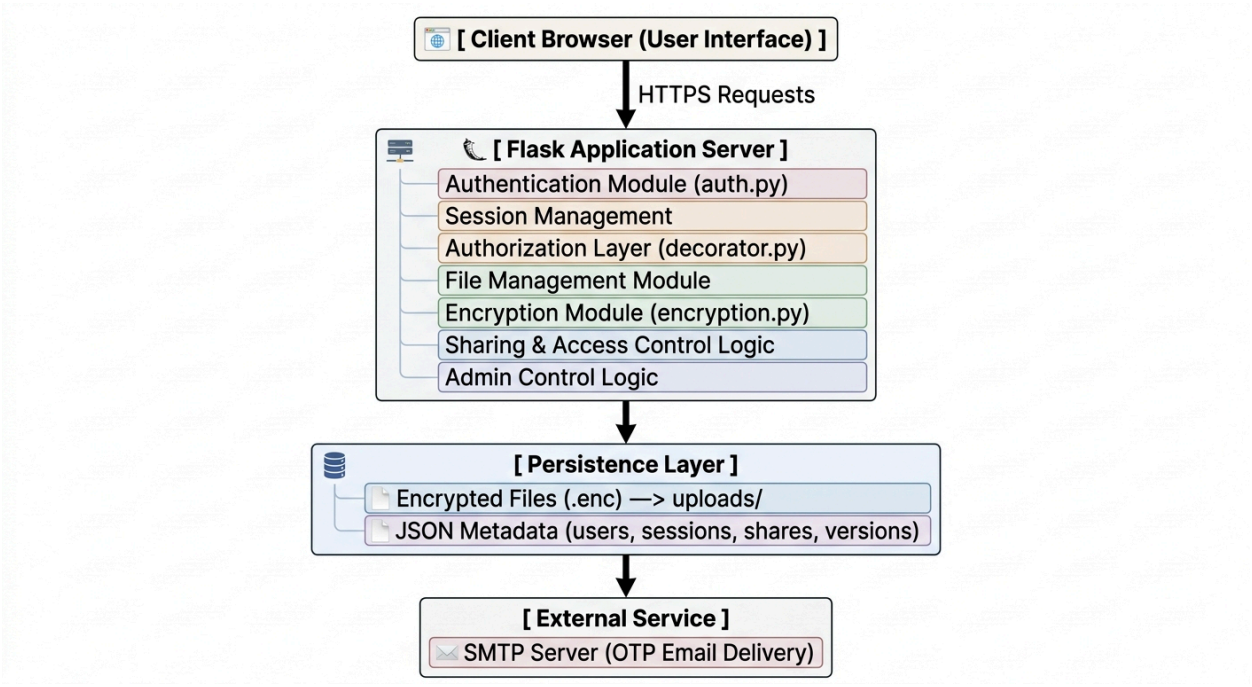
Security Design Document

SecureDocs is a web-based document management and sharing system designed with a strong emphasis on security, access control, and data protection. The system supports multiple user roles—administrators, users, and guests—and enables secure file upload, versioning, and controlled sharing.This document provides a comprehensive security design analysis of SecureDocs, including system architecture, threat modeling, implemented security controls, and data protection mechanisms.

A. Architecture Overview (10 points)

A1. System Architecture Diagram



The SecureDocs system follows a layered client-server architecture in which the client browser serves as the presentation layer, the Flask application server serves as the application and security enforcement layer, the storage subsystem serves as the persistence layer, and the SMTP service supports external recovery operations. Users interact with the system through browser-based interfaces defined in templates such as login.html, register.html, dashboard.html, upload.html, and share.html. These interfaces collect user input and submit requests involving authentication, file uploads, sharing actions, downloads, and password reset requests.

All trust decisions occur within the Flask application server rather than within the client. This architectural decision is critical because it prevents users from bypassing security controls through client-side manipulation. The application server integrates several security-focused modules, including the authentication logic associated with auth.py, the authorization protections associated with decorator.py, encryption protections implemented in encryption.py, sharing controls, file management logic, and

administrative approval functions. By separating these responsibilities into modules, the architecture improves maintainability while also supporting layered security enforcement.

The storage layer separates encrypted file content from metadata, which reduces the impact of compromise affecting any single component. Encrypted documents are stored as protected .enc files in the upload directory, while user records, sessions, and sharing relationships are maintained separately through JSON-based metadata. This separation improves confidentiality, supports integrity controls, and limits the likelihood that compromise of one storage element results in full system exposure.

The architecture also isolates password recovery functions through the external SMTP service defined in config.py. By separating OTP delivery from the core application itself, SecureDocs reduces coupling between authentication and recovery logic while supporting out-of-band identity verification. Overall, this layered design supports separation of concerns, centralized security decision-making, reduced attack surface, and defense-in-depth principles.

Code Reference: The architecture shown is implemented primarily through app.py (application routing and control logic), auth.py (authentication logic), decorator.py (authorization enforcement), encryption.py (document encryption), and config.py (security configuration parameters).

A2. Data Flow Diagrams

A2.1 User Authentication Flow

User -> Login Form -> Flask Route (/login)
-> Credential Validation (username/email + password)
-> Account Status Check (approved/locked)
-> Session Token Generation (secrets.token_urlsafe)
-> Session Stored (sessions.json)
-> Redirect to Dashboard

Security Enforcement:

Password validation occurs server-side, Account lockout enforced using **MAX_FAILED_ATTEMPTS,** Session tokens are **cryptographically secure,** session expiration enforced using **SESSION_TIMEOUT**

Justification:

Using **secrets.token_urlsafe(32)** ensures:

High entropy session tokens and also resistance to brute-force guessing

Code Reference: This authentication flow is implemented through login handling and session creation logic in app.py and credential verification functions in auth.py.

A2.2 File Upload Flow

- User ->Upload Form -> Flask Route (/upload)
- ->Role Check (admin/user only)
- -> File Validation (extension + size)

- -> Filename Sanitization
- Encryption (Fernet)
- Stored as .enc file
- Version Incremented

Metadata Updated

Security Enforcement:

- Only authorized roles can upload
- File type restricted via ALLOWED_EXTENSIONS
- Files encrypted before disk write
- Versioning prevents overwrite attacks

Justification:

Encrypting files **before storage** ensures that there is confidentiality even if the filesystem is compromised. Additionally, there is no plaintext sensitive data exposure.

Code Reference: This upload flow is implemented through upload route protections in app.py, validation controls defined in config.py, and encryption protections implemented in encryption.py.

A2.3 File Access / Download Flow

User -> Download Request -> Flask Route (/download) -> Session Validation-> Authorization Check (ownership/shared) -> File Retrieval (.enc) -> Decryption (Fernet) -> File Sent to User

Security Enforcement:

Access checked against sharing metadata, decryption occurs only after authorization and files never stored or transmitted in plaintext on disk

Code Reference: This access flow is implemented through download authorization logic in app.py and decryption protections in encryption.py.

A2.4 File Sharing Flow

Owner -> Share Request -> Flask Route (/share) -> Validate Target User/Guest -> Update Sharing Metadata (shares.json) -> Access Granted

Security Enforcement:

Only the file owner or admin can share, and access control enforced during every download request.

Code Reference: This sharing flow is implemented through sharing logic and access metadata enforcement in app.py and shares.json management logic.

A2.5 Password Reset (OTP Flow)

User -> Reset Request -> Validate Username + Email -> Generate OTP -> Send via SMTP -> Store OTP (temporary) -> User submits OTP -> Validate OTP + Expiry-> Allow password reset

Security Enforcement:

OTP expires after 10 minutes, the OTP is tied to user identity, and it also prevents unauthorized resets

Code Reference: This recovery flow is implemented through OTP generation, SMTP delivery, and verification logic in app.py and security parameters defined in config.py.

Overall Data Flow Justification

All flows enforce:**Authentication first,Authorization before action,Validation before processing, and Encryption before storage.** This follows the principle of **defense in depth**.

A3. Component Descriptions

The authentication component implemented through auth.py is responsible for login processing, session creation, session expiration, and failed login tracking. Session records stored in sessions.json include contextual information such as user identifiers, timestamps, IP addresses, and user-agent information, which strengthens protection against session misuse and supports contextual validation.

Authorization protections implemented through decorator.py enforce role-based access restrictions for administrators, users, and guests. These protections ensure that sensitive routes such as administrative functions, upload operations, and restricted management actions cannot be accessed without proper privileges. Because authorization checks occur server-side, they remain effective even if client-side controls are manipulated.

The file management component handles uploads, downloads, versioning, and deletion protections. Versioning using the FILE_VERSION_FORMAT convention strengthens integrity protections by preventing unsafe overwrites and supporting controlled recovery of previous document versions. This component works closely with the encryption module, implemented through encryption.py, which provides confidentiality and integrity protections using Fernet symmetric encryption. Because encryption occurs before storage and decryption occurs only after authorization, confidentiality protections remain integrated directly into the file lifecycle.

The sharing module manages controlled document access through sharing relationships stored in metadata and enforced during access decisions. This supports least-privilege principles by granting access only to explicitly authorized users. Administrative functions support additional controls by managing approval workflows, overseeing users, and enforcing system-wide management decisions.

Finally, the storage layer combines encrypted file persistence with separated metadata storage, reducing the likelihood that compromise of a single component would expose both protected data and authorization structures.

Together, these components form an integrated architecture in which authentication, authorization, encryption, storage protection, and administrative oversight operate as coordinated security controls rather than isolated features.

A4. Technology Stack Justification

The technology stack was selected not only for development simplicity, but also for its security properties.

Flask was selected because its minimal architecture reduces attack surface while allowing explicit implementation of custom controls such as RBAC, session validation, and encryption logic. This improves auditability by keeping security mechanisms visible in application code.

Python was selected because its mature security ecosystem supports integration of established libraries such as the **cryptography** package for Fernet encryption, secure random generation using the **secrets** module, and standard libraries supporting secure file handling and input validation.

The use of Jinja2 templates, visible throughout **login.html, register.html, dashboard.html, upload.html,** and **share.html,** also supports security by providing automatic HTML escaping, reducing exposure to cross-site scripting risks if used properly.

JSON metadata storage was selected for implementation simplicity and transparency. Although a database would provide stronger concurrency control and scalability, JSON storage allows easier inspection and verification of stored authorization mappings, sessions, and ownership records during development and testing.

The local filesystem was selected as the storage layer because encrypted files remain under direct application control rather than relying on external storage dependencies. This simplifies access control integration, since the application mediates all file access.

The SMTP integration supports secure out-of-band recovery operations. By separating password reset delivery from the core application logic, the design reduces coupling between authentication and recovery mechanisms.

B. Threat Model (10 points)

B1. Asset Identification

In addition to primary assets such as credentials and uploaded documents, SecureDocs also protects supporting security assets including encryption keys, session state information, account approval status, version histories, and sharing relationships.

The encryption key used by **encryption.py** is itself a critical asset because compromise of the key could undermine confidentiality protections protecting stored files.

Administrative privileges also represent a high-value asset because compromise of an administrator account could affect approval workflows, user management, and system-wide document access.

Because security assets include not only data but also trust relationships and privilege structures, the threat model considers both direct data compromise and control compromise.

B2. Threat Enumeration

Threats to SecureDocs include external attacks, insider misuse, and operational threats.

External threats include brute-force attacks, credential stuffing, malicious uploads, session hijacking, and request forgery.

Insider threats include abuse of legitimate sharing permissions, misuse of administrative privileges, and unauthorized data access by otherwise authenticated users.

Operational threats include accidental deletion, encryption key loss, configuration errors, and dependency compromise involving SMTP services or TLS certificates.

Considering both malicious attacks and operational failures strengthens the threat model by recognizing that security risks may arise from both attackers and system conditions.

B3. Vulnerability Assessment

Unlike threats, which represent potential attacks against the system, vulnerabilities represent weaknesses within the system design or implementation that may be exploited.

Several vulnerabilities exist within SecureDocs.

First, the account lockout mechanism is enforced at the account login level rather than per source IP address. While this limits repeated login attempts against a single account, distributed brute-force attempts from multiple origins may partially bypass this control.

Second, uploaded files are validated primarily through extension restrictions and file size checks. While these controls reduce exposure to malicious uploads, extension validation alone may not reliably detect disguised malicious content without deeper MIME inspection or malware scanning.

Third, SecureDocs mitigates Cross-Site Request Forgery partially through SameSite=Strict session cookie protections configured in app.py, which restrict cross-site transmission of authenticated session cookies. However, the system does not currently implement dedicated per-request anti-CSRF tokens, which would provide stronger defense-in-depth protections.

Fourth, encryption key management currently depends on locally managed server-side key storage. While the encryption key is separated from encrypted files, absence of automated key rotation or dedicated key vault protections introduces operational risk.

Finally, JSON-based metadata storage simplifies implementation but provides weaker concurrency protections and integrity guarantees than a hardened database-backed storage model.

These vulnerabilities are partially mitigated through layered controls, but represent areas for future security improvement.

B4. Attack Scenarios

An attacker registered as a standard user may attempt to manipulate requests, modify parameters, or exploit missing authorization checks to perform actions reserved for administrators.

This scenario is particularly relevant because approval workflows, user management functions, and file administration capabilities represent high-value privilege targets.

This attack is mitigated through role validation logic associated with `decorator.py`, route protections, and server-side authorization checks.

Including privilege escalation strengthens the threat model because authorization failures are a common application security risk.

B5. Risk Prioritization

Risk prioritization is based on both likelihood and impact.

Credential theft is prioritized as high risk because compromise of authentication may expose both user data and broader system access.

File leakage is also high risk because document confidentiality is a core security objective.

CSRF is prioritized as moderate-to-high risk due to potential impact on integrity and access-control decisions, particularly for privileged administrative actions.

Key compromise represents an additional high-impact risk because it could undermine encryption protections protecting all stored documents.

Prioritization focuses first on risks capable of undermining core security goals: confidentiality, integrity, and access control.

C. Security Controls (10 points)

C1. Authentication Control

~Control Description

Authentication in SecureDocs is designed to ensure that only legitimate users can access protected resources. The control enforces identity verification during login, protects accounts from unauthorized access attempts, supports password recovery through OTP verification, and establishes trusted authenticated sessions after successful credential validation. This control protects against credential abuse, unauthorized account access, and session misuse.

Authentication mechanisms are reflected in the login workflow implemented through **login.html,** the registration process in **register.html,** the password reset workflow in **resetpassword.html, verify_otp.html,** and **set_new_password.html,** and security parameters defined in **config.py.**

Implementation Approach

The authentication mechanism validates submitted credentials before establishing a server-side session. Session duration is restricted through:

SESSION_TIMEOUT = 1800

Authentication protections are reinforced through lockout controls defined in:

MAX_FAILED_ATTEMPTS = 5 and **ACCOUNT_LOCK_TIME = 15 * 60**

These controls limit repeated failed login attempts and temporarily disable access after threshold violations.

OTP-based password reset adds a second identity verification step using:

OTP_EXPIRY_SECONDS = 600

OTP delivery occurs through the SMTP configuration defined in **config.py** using Gmail SMTP.

Together, these controls combine credential verification, session management, lockout protection, and secondary identity verification.

Testing Methodology

Authentication controls were tested through successful and unsuccessful login attempts, incorrect password submissions, repeated failed login attempts to trigger lockout thresholds, expiration testing for inactive sessions, and OTP verification tests involving valid, invalid, and expired codes.

Additional testing verified that protected routes redirect unauthenticated users and that authenticated sessions are required before dashboard access is granted.

Known Limitations

The lockout mechanism applies at the account level rather than per source IP, meaning distributed brute-force attempts from multiple origins may partially bypass this control.

OTP is email-dependent and therefore assumes security of the email provider. Compromise of the email account could weaken reset protections.

Session protection also assumes strong protection of session tokens against theft.

Mitigation Strategies

These limitations are partially mitigated through strong session randomness, session expiration, account lockout controls, and OTP expiration restrictions.

Future improvements would include multi-factor authentication, CAPTCHA protections, and stronger session-binding mechanisms.

C2. Session Management

Control Description

Session controls protect authenticated sessions against hijacking, persistence abuse, and unauthorized reuse.

Implementation Approach

Sessions are created after authentication and expire based on: **SESSION_TIMEOUT**

Session identifiers use secure randomness through: **secrets.token_urlsafe(32)**

Sessions are managed via a server-side session manager and persisted in a JSON-based session store. **(sessions.json)**

Testing Methodology

Testing included timeout validation, session expiration testing, logout verification, and attempts to reuse expired sessions.

Known Limitations

Session protection assumes token secrecy and secure client handling.

Current controls may not fully bind sessions to device characteristics.

Mitigation Strategies

Mitigations include token randomness, timeout enforcement, and future support for secure cookie flags and stronger session binding.

C3. Access Control (RBAC)

Control Description

Access control enforces role restrictions across administrators, users, and guests. It prevents unauthorized users from accessing restricted resources, limits administrative functions to privileged roles, and restricts file operations based on ownership and sharing permissions.

These controls are visible in **dashboard.html, base.html, share.html, users.html,** and the route protection logic associated with **decorator.py.**

Implementation Approach

Role-based restrictions are enforced through protected routes and role validation checks such as:

@require_role('admin','user')

Template-level authorization logic further restricts access through conditions such as:

current_user_role() == 'admin'and: **current_user_role() in ['admin','user']**

File access permissions are validated through ownership checks, shared access mappings, and administrative override permissions.

Testing Methodology

Testing involved logging in under each role and verifying allowed and denied operations. Guest users were tested to confirm upload restrictions. Standard users were tested to confirm inability to access administrative controls. File-sharing permissions were tested through shared and unshared access attempts.

Authorization bypass attempts were also tested through direct URL access to restricted routes.

Known Limitations

Access enforcement assumes route protections are consistently applied. Misconfigured or unprotected routes could create exposure.

Template-level visibility restrictions do not replace server-side authorization and must remain synchronized with backend checks.

Mitigation Strategies

Mitigation includes layered authorization enforcement at both route and application logic levels, regular authorization testing, and code review to verify route protections remain consistently applied.

Future improvements include centralized policy enforcement and attribute-based access controls.

C4. File Encryption

Control Description

File protection safeguards uploaded documents from unauthorized disclosure, tampering, or improper deletion.

Protection covers upload validation, encryption, access restriction, and version control protections.

Relevant controls appear in **upload.html,** **dashboard.html,** **config.py,** and **encryption.py.**

Implementation Approach

Files are restricted using: **ALLOWED_EXTENSIONS** and size limits enforced through: **MAX_CONTENT_LENGTH**

Files are encrypted before storage through Fernet encryption and stored as protected **.enc** files.

Version control prevents accidental overwrites and supports integrity management.

File validation includes extension and MIME-type checks/verifications to reduce the risk of malicious file uploads.

Testing Methodology

Testing included valid and invalid file uploads, oversized file rejection tests, encrypted storage verification, file retrieval testing, and version management testing.

Deletion tests verified removal behavior for authorized users.

Known Limitations

Extension validation alone may not detect disguised malicious files.

Local filesystem storage lacks advanced enterprise protections.

Secure deletion protections may depend partly on filesystem behavior.

Mitigation Strategies

These risks are mitigated through encryption, size restrictions, access controls, and version controls.

Future improvements include malware scanning, and hardened storage backends.

C5. Rate Limiting

Control Description

Rate limiting protects against abuse, automated attacks, and excessive request activity.

Implementation Approach

Request throttling is configured through:

MAX_ATTEMPTS_PER_MINUTE = 10 and WINDOW_SECONDS = 60

These controls restrict request frequency, and Rate limiting is applied to login attempts per IP address to mitigate brute force attacks.

Testing Methodology

Testing involved repeated requests exceeding thresholds and verifying restriction behavior.

Known Limitations

Per-account controls may not fully address distributed attacks, for now it just addresses login attempts.

Mitigation Strategies

Future improvements include IP-level throttling and reverse proxy protections.

C6. OTP Security

Control Description
OTP security protects the password reset process from unauthorized account recovery by requiring a temporary one-time verification code tied to a specific user identity and valid only for a limited time.

Implementation Approach
OTP protections are enforced through:

OTP_EXPIRY_SECONDS = 600

OTP values are generated during password reset, transmitted through the SMTP service, and validated before allowing password changes through the verify_otp and set_new_password workflow.

Testing Methodology
Testing included valid OTP verification, invalid OTP rejection, expired OTP rejection, and repeated incorrect OTP attempts.

Known Limitations
OTP security depends partly on the security of the user’s email account. Compromise of the email provider could weaken recovery protections.

Mitigation Strategies
Risk is mitigated through short OTP expiration windows, user identity validation before OTP generation, and future support for multi-factor authentication.

C7.Monitoring and Auditing

Control Description

Monitoring controls provide visibility into failed authentication attempts, user approvals, and security-relevant events.

Implementation Approach

Monitoring is supported through failed login counters, approval workflows in **dashboard.html**, user administration in **users.html**, and audit-related metadata maintained in **audit.json**.

Testing Methodology

Testing verified visibility of approval states, failed attempt tracking, and administrative review functionality.

Known Limitations

The system does not yet implement centralized audit logging or intrusion detection.

Mitigation Strategies

Preventive controls partially compensate for limited monitoring.

Future enhancements include full audit logs and alerting.

C8. Input Validation (XSS and Path Traversal Protection)

Control Description
Input validation protects SecureDocs against injection attacks, malicious input manipulation, path traversal attempts, and cross-site scripting risks.

Implementation Approach
Input validation protections are supported through multiple mechanisms.

Jinja2 template auto-escaping protects dynamic output rendered in templates such as login.html, dashboard.html, share.html, and users.html.

Filename protections use secure_filename() to sanitize uploaded filenames before filesystem operations.

Extension whitelisting through ALLOWED_EXTENSIONS restricts disallowed file types.

Additional validation is applied through controlled username, email, and form input processing before security-sensitive operations are performed.

Testing Methodology
Testing included attempted XSS payload submission using: **<script>alert(1)</script>**

in form inputs, attempted path traversal using: **../../../etc/passwd**

in filename-related inputs, and invalid input testing across registration, sharing, and upload workflows.

Known Limitations
Auto-escaping protections may be weakened if unsafe template rendering practices are introduced. Extension validation alone does not fully inspect malicious file content.

Mitigation Strategies
Mitigations include template auto-escaping, filename sanitization, whitelist validation, and future support for MIME inspection and malware scanning.

D. Data Protection (10 points)

D1. Data Classification

SecureDocs processes multiple data classes requiring different levels of protection.

Uploaded documents represent highly sensitive protected data and are safeguarded through encryption in **encryption.py**, storage controls associated with **upload.html**, and access restrictions enforced through sharing and authorization logic.

Authentication data, including credentials, session identifiers, OTP codes, and failed login counters defined in **config.py**, also receive elevated protection because compromise could enable unauthorized access.

Authorization metadata, including ownership mappings, sharing permissions, and approval status reflected in **dashboard.html**, **share.html**, and **users.html**, are classified as protected internal security data because compromise could weaken access control.

Data classification supports proportional protection based on sensitivity.

D2. Encryption Methods

SecureDocs protects stored documents through Fernet symmetric encryption implemented through **encryption.py.**

Files are encrypted before being written to persistent storage and stored as protected **.enc** files using the versioning format defined in:

FILE_VERSION_FORMAT = "{filename}_v{version}.enc"

Fernet was selected because it provides both confidentiality and integrity protection.

Transport security is supported through TLS configuration references in config.py:

SSL_CERT = "cert.pem" SSL_KEY = "key.pem"

These settings support protection of credentials, sessions, and file transfers during transmission.

D3. Key Management

Encryption keys are managed separately from encrypted documents and are not exposed through client-side code or templates.

Access to keys is limited to server-side processes associated with encryption.py.

Current limitations include local key storage and lack of automated rotation.

Future improvements include protected key vaults, environment variable storage, and periodic re-encryption procedures.

D4. Secure Deletion Procedures

Secure deletion is supported through version deletion functionality reflected in dashboard.html.

Deletion requires removal of both encrypted files and associated metadata.

Deleting only one component would create residual exposure or integrity issues.

A limitation is that current deletion may rely partly on filesystem behavior rather than guaranteed overwrite procedures.

This is partially mitigated because files remain encrypted even prior to deletion.

D5. Backup and Recovery Procedures

SecureDocs supports recovery through preservation of encrypted file versions and separated metadata storage for users, shares, sessions, and version records. Because encrypted files and metadata are maintained separately, recovery can be performed by restoring both protected document data and associated authorization mappings.

Versioning provides limited recovery support by preserving prior document states in addition to supporting integrity protections.

Current limitations include the absence of automated scheduled backups, offsite replication, and formal disaster recovery procedures.

These limitations could be mitigated in future versions through encrypted automated backups, secure offsite storage, periodic recovery testing, and documented disaster recovery procedures.